

Perfect. Before writing any code, let's behave like an AI PM would.

We are **not building a side project**.

We are building something that should look like an internal product used by companies like OpenAI, Anthropic, Cursor, Perplexity, or Glean to decide whether an AI feature is ready for production.

Project Name

AI Release Intelligence Platform (ARIP)

Tagline:

Ship AI Features with Confidence.

Alternative names (we can finalize later):

- EvalForge
- PromptLab
- ShipAI
- EvalOps
- Prompt Intelligence
- AI Launchpad
- ReleaseIQ
- PromptLens

I actually prefer **AI Release Intelligence Platform** because it immediately conveys product value rather than just technology.

Phase-0 Goals

At the end of Phase-0 we should have:

- ✓ Product Vision
- ✓ Problem Statement
- ✓ Personas
- ✓ JTBD
- ✓ MVP Scope
- ✓ Tech Stack

- ✓ Development Environment
- ✓ Git Repository
- ✓ Cursor/Codex Rules
- ✓ Initial Project Structure
- ✓ First UI Wireframes
- ✓ Initial PRD
- ✓ Prompt Library
- ✓ Development Roadmap

Notice that **we still haven't written application code.**

This mirrors how a PM initiates a product.

Folder Structure

I recommend starting with this:

```
AI-Release-Intelligence-Platform/
```

```
|
├── docs/
│   ├── PRD.md
│   ├── Roadmap.md
│   ├── UserFlows.md
│   ├── Personas.md
│   ├── Metrics.md
│   ├── Architecture.md
│   └── PromptLibrary.md
├── frontend/
├── backend/
├── database/
├── prompts/
├── assets/
├── .cursor/
|
```

```
├── README.md
└── TODO.md
```

Notice:

Documentation is a first-class citizen.

Hiring managers appreciate this.

GitHub Repository

Suggested repository:

ai-release-intelligence-platform

Description:

An AI Product Management platform for evaluating, comparing, and safely releasing l

Initial Tech Stack

I recommend **not overengineering**.

Frontend

- Next.js
- React
- TypeScript
- Tailwind

Backend

- FastAPI

Database

- PostgreSQL

ORM

- Prisma

Authentication

- None (Phase-0)

Deployment

- Vercel
- Railway/Supabase later

LLM

- OpenAI initially

Initial Documentation

Create these empty docs immediately.

PRD.md

Roadmap.md

Architecture.md

Metrics.md

PromptLibrary.md

UserFlows.md

Personas.md

First PRD Sections

Don't write everything.

Just headings.

Vision

Problem

Goals

Non Goals

Target Users

Pain Points

User Stories

Success Metrics

MVP

Future Scope

Risks

Open Questions

Development Principles

This becomes your north star.

Every feature must answer:

Why does an AI PM need this?

How does this help ship AI safely?

How is success measured?

What decision becomes easier?

Cursor/Codex Rules

Create:

`.cursor/rules.md`

Suggested content:

Markdown

You are a senior AI engineer and staff product engineer.

This project prioritizes:

- readability
- modular architecture
- production-quality code
- reusable components
- clear naming
- typed interfaces
- minimal dependencies

Always explain architectural tradeoffs.

Prefer composition over inheritance.

Never introduce unnecessary complexity.

Write code suitable for production.

Include comments explaining WHY instead of WHAT.

Favor maintainability over cleverness.

Whenever adding a feature:

- explain the user problem
- explain design choices
- explain tradeoffs

This alone dramatically improves AI coding assistants.

Prompt Library

We'll maintain prompts separately.

Example:

prompts/

prompt-evaluator.md

prompt-reviewer.md

ui-generator.md

architecture-review.md

prd-generator.md

bug-review.md

feature-planner.md

This becomes reusable across future projects.

Initial Prompt

This is the first prompt I'd give Codex.

Prompt 1

You are my Staff Software Engineer.

We are building an AI Release Intelligence Platform.

This is NOT another chatbot.

Our users are AI Product Managers and AI Engineers.

The platform helps teams decide whether an AI feature is ready for production.

Primary capabilities:

- Prompt experimentation
- AI evaluation
- Prompt comparison
- Release intelligence
- Human review
- Cost analysis
- Latency analysis
- Regression detection

The application should prioritize modular architecture, clean code, scalability, ar

Before writing code:

1. Propose the overall architecture.
2. Identify core modules.
3. Suggest database entities.
4. Suggest API boundaries.
5. Explain tradeoffs.
6. Wait for approval before implementation.

Prompt 2

For UI generation.

You are a Senior Product Designer.

Design a modern SaaS dashboard for an AI Release Intelligence Platform.

Users:

- AI Product Managers
- AI Engineers

The design should resemble Linear, Vercel, Notion, and Cursor.

The product should feel enterprise-ready rather than experimental.

Generate:

- Information architecture
- Navigation
- Dashboard layout
- Feature hierarchy
- User flows

Focus on usability over visual effects.

Prompt 3

For Product Thinking.

Act as an AI Product Director.

Critique every feature proposal using these questions:

- Who benefits?
- What user problem does it solve?
- How often is this problem encountered?
- Can the user accomplish the same task without this feature?
- How will success be measured?
- Should this feature be part of the MVP?
- What risks does it introduce?
- Would OpenAI or Anthropic build this feature?

Reject unnecessary complexity.

Prioritize features that improve release confidence and product decision-making.

Phase-0 Milestone

By the end of this phase, we should have:

- A Git repository initialized with the agreed structure.
- Documentation scaffolding in place.
- Coding assistant rules configured.
- Three foundational prompt templates ready.
- A clear product vision and MVP definition.

Only then do we move into implementation. This sequence mirrors how strong product teams reduce ambiguity before investing engineering effort.

One suggestion to increase portfolio value

Instead of keeping the repository purely code-focused, create a top-level `portfolio/` folder from day one. It can eventually hold your PRD, design decisions, experiment logs, screenshots, metrics, demo script, and interview talking points. By the time the project is complete, you'll have not just a working application but a complete product case study that hiring managers can review end-to-end.

I have given these 3 prompts to the codex, I don't think the implementation has started yet.
I have created the git repo.
Let me know the next steps to start next phase or implementation.

Perfect. This is actually the right place to pause and make one important adjustment.

Those three prompts were intended to **align the AI assistant**, not to build the product. If Codex started implementing immediately, we'd likely end up with a generic SaaS dashboard instead of a differentiated AI PM product.

At this point, **you don't need more planning—you need a clear execution loop.**

Our Build Philosophy

We're going to build **vertically**, not horizontally.

❌ Don't build:

- Complete frontend
- Complete backend
- Complete database

Instead build one complete feature at a time.

Example:

Prompt Playground

- Database
- API
- UI
- Testing
- Polish

Then move to the next feature.

This is how good product teams work.

Revised Phase Plan

Phase 1 (Week 1)

Instead of calling it "implementation," I'd call it:

Build the Core Product

Goal:

A PM can compare two prompts against one input.

Nothing more.

If you finish this, you'll already have something demoable.

Feature 1 — Prompt Workspace

Imagine opening the application.

You see:

+-----+

Prompt A

Prompt B

Model

GPT-5

Input

Compare

Result A

Result B

Latency

Token Usage

Estimated Cost

That's your first milestone.

Not authentication.

Not dashboards.

Not analytics.

Those come later.

What Needs to Be Built

Database

Start with only four tables.

Prompt

EvaluationRun

Response

Model

Nothing else.

Backend

Only five endpoints.

```
POST /prompts

GET /prompts

POST /compare

GET /run/:id

DELETE /prompt
```

That's it.

Frontend

Only four pages.

```
Home

Prompt Playground

Run History

Settings
```

Nothing more.

Your Next Prompt to Codex

This is the first implementation prompt I'd use.

```
You are the Lead Software Engineer for this project.

We have completed product discovery.

Now begin implementation.

Our first milestone is a fully functional Prompt Playground.

Requirements:
```

Users should be able to:

- Create Prompt A
- Create Prompt B
- Choose a model
- Enter one input
- Execute both prompts
- Display results side by side
- Show:
 - latency
 - token usage
 - estimated cost
- Save every comparison to PostgreSQL.

Do NOT build authentication.

Do NOT build analytics.

Do NOT build evaluation datasets.

Do NOT build dashboards.

Implement only what is required for this milestone.

Before writing code:

1. Create the folder structure.
2. Define the database schema.
3. Define API contracts.
4. Explain architecture.
5. Then implement incrementally.
6. Commit after each logical milestone.

Then Don't Let Codex Run Wild

This is where many AI-built projects go wrong.

Instead, work in this loop:

Codex proposes

↓

You review

↓

Accept

↓

Codex builds ONE feature

↓

Test

↓

Commit

↓

Next feature

Don't ask it to build the entire application in one go.

First GitHub Milestones

I'd create these milestones:

M1

Project Setup

M2

Database Schema

M3

Backend APIs

M4

Prompt Playground UI

M5

LLM Integration

M6

Save Run History

M7

Polish MVP

Each milestone should ideally correspond to one or a few commits.

What You Should Personally Own (This Is the AI PM Part)

This is the most important distinction.

Codex is your engineer. You're the PM.

For every feature, you should explicitly define:

- **User problem:** What pain point does this solve?
- **Success metric:** How will we know the feature works?
- **Acceptance criteria:** What behavior is required before it's "done"?
- **Trade-offs:** What are we deliberately not building yet?
- **Release decision:** Is this feature ready for the next milestone?

This gives you authentic material to discuss in interviews because you'll be able to explain *why* each feature exists, not just *how* it was implemented.

I Suggest One More Change Before Coding

Rather than asking Codex to create the entire Prompt Playground in one prompt, split the work into **small, reviewable implementation stories**.

For example:

1. **Story 1:** Bootstrap the repository and folder structure.
2. **Story 2:** Design the PostgreSQL schema with Prisma.
3. **Story 3:** Implement prompt CRUD APIs.
4. **Story 4:** Build the Prompt Playground UI.

5. Story 5: Integrate the LLM provider and compare two prompts.

6. Story 6: Persist run history and display previous comparisons.

Each story should take roughly **30-90 minutes** to implement and review. This approach gives you cleaner commits, easier debugging, and a much stronger development narrative for your portfolio than a single massive AI-generated code drop.

Today 8:35 AM

Can you draft a brief and concise plan for this project phase by phase? Clearly mention what we are achieving in each phase wrt AI evaluation. What the user will do and what he can expect from the feature (value)? Currently, I am testing the outputs of two models based on prompts for phase-1.

Yes. I would actually organize the roadmap around **AI evaluation maturity**, not technical components. That makes the product story much stronger in interviews because every phase answers a progressively more sophisticated AI product question.

AI Evaluation Studio Roadmap

Phase 1 — Prompt Playground & Model Comparison (Current MVP)

Objective

Enable AI teams to quickly compare prompts and models before making changes.

User does

- Enter an input/query.
- Create multiple prompt versions.
- Select one or more LLMs.
- Run the same input against each combination.
- View outputs side by side.

User gets

- Side-by-side response comparison.

- Latency.
- Token usage.
- Estimated cost.
- Basic run history.

Value

"Which prompt/model gives the best output for this specific query?"

This is **manual experimentation**, similar to how many teams currently work.

Phase 2 — Dataset-Based Evaluation

Objective

Move from testing one example to evaluating an entire use case.

User does

- Upload or create an evaluation dataset.
- Define expected outputs or reference answers.
- Run prompts/models across all test cases.

User gets

- Aggregate quality score.
- Success/failure rate.
- Per-test-case results.
- Failure analysis.

Value

"Does this prompt work consistently across real user scenarios?"

This introduces **repeatable evaluation** instead of anecdotal testing.

Phase 3 — AI Judge & Evaluation Metrics

Objective

Automate response quality assessment.

User does

- Choose evaluation criteria (accuracy, completeness, tone, etc.).

- Run evaluations using an LLM-as-a-judge.

User gets

- AI-generated quality scores.
- Reasoning behind scores.
- Hallucination indicators.
- Structured evaluation reports.

Value

"How good is this response without manually reviewing every output?"

This is where the product becomes a genuine **AI evaluation platform**.

Phase 4 — Experiment Tracking & Versioning

Objective

Track improvements over time.

User does

- Save prompt versions.
- Compare experiments.
- Add notes and release tags.

User gets

- Prompt version history.
- Experiment comparison.
- Performance trends.
- Best-performing prompt identification.

Value

"Did this change actually improve the product?"

This introduces **experiment management**, a core AI PM responsibility.

Phase 5 — Human Review Workflow

Objective

Blend AI evaluation with human judgment.

User does

- Review flagged responses.
- Rate outputs.
- Override AI evaluations.
- Label failure reasons.

User gets

- Human review queue.
- Reviewer agreement metrics.
- Feedback dataset.
- High-quality labeled examples.

Value

"Some decisions need humans. Let's capture that feedback systematically."

This reflects how production AI teams build trust and improve quality.

Phase 6 — AI Product Analytics

Objective

Turn evaluations into product insights.

User does

- Monitor evaluation trends.
- Filter by model, prompt, or use case.
- Investigate recurring failures.

User gets

- Quality trends.
- Cost trends.
- Latency trends.
- Failure distribution.
- Prompt leaderboard.

Value

"Where should we invest next to improve our AI product?"

Now the platform supports **product decision-making**, not just testing.

Phase 7 — Release Intelligence

Objective

Decide whether an AI feature is ready for production.

User does

- Compare a release candidate against the current production version.
- Review quality gates.
- Approve or reject releases.

User gets

- Regression detection.
- Release readiness score.
- Go/No-Go recommendation.
- Release summary.

Value

"Can we confidently ship this AI change?"

This is the culmination of the platform: helping teams make informed release decisions.

Visual Progression

```
graph TD; P1[Phase 1  
Compare Outputs] --> P2[Phase 2  
Evaluate on Datasets]; P2 --> P3[Phase 3  
Automatically Score Responses]; P3 --> P4[Phase 4  
Track Experiments & Versions]; P4 --> P5[Phase 5  
Add Human Feedback]; P5 --> P6[Phase 6  
Analyze Trends & Insights];
```

Phase 1
Compare Outputs

↓

Phase 2
Evaluate on Datasets

↓

Phase 3
Automatically Score Responses

↓

Phase 4
Track Experiments & Versions

↓

Phase 5
Add Human Feedback

↓

Phase 6
Analyze Trends & Insights

↓

Why this roadmap is compelling

Each phase introduces a distinct AI PM competency:

Phase	AI PM Skill Demonstrated	JD Coverage
1	Prompt engineering, model comparison	Very High
2	Evaluation strategy, benchmark design	Very High
3	AI quality metrics, LLM-as-a-judge	High
4	Experimentation and iteration	Very High
5	Human-in-the-loop workflows	High
6	AI analytics and product metrics	High
7	Release management and AI governance	Very High

This progression tells a cohesive story: you didn't build a collection of unrelated features—you built a platform that evolves from simple experimentation into a production-ready AI evaluation and release system, mirroring how mature AI organizations develop and improve their products over time.

ChatGPT can make mistakes. Check important info.